

FIAP

Global Solution 2026 — 1º Semestre

Indústria Espacial: O Código que Move o Universo

Backlog do Produto

Orbital Memory

A caixa-preta das cidades em situações de desastre

Engenharia de Software — 1º ano — Semipresencial

Estudante: Ana Carolina Ramalho de Matos

RM: 571219

Porto Alegre
2026

Sumário

Capa.....	1
Sumário.....	2
3. Resumo executivo.....	3
4. Introdução.....	4
5. Visão geral do produto.....	5
6. Requisitos funcionais.....	6
7. Requisitos não funcionais.....	8
8. Regras de negócio.....	10
9. Épicos do sistema.....	11
Épico 1 — Monitoramento urbano.....	11
Épico 2 — Alertas emergenciais.....	11
Épico 3 — Histórico de eventos.....	11
Épico 4 — Redundância orbital.....	12
Épico 5 — Análise computacional de risco.....	12
Épico 6 — Estação física simulada.....	12
10. Histórias de usuário.....	13
11. Priorização do backlog.....	15
Backlog priorizado.....	15
12. Jornada do usuário.....	17
13. Kanban do projeto.....	18
Organização das tarefas por disciplina.....	18
14. Critérios de aceite.....	20
15. Conclusão.....	21
16. Referências.....	22

3. Resumo executivo

O presente documento apresenta o backlog do produto **Orbital Memory**, uma plataforma de preservação e monitoramento de dados críticos urbanos voltada para situações de desastre natural. O sistema foi pensado como uma “caixa-preta urbana”, capaz de registrar informações importantes antes, durante e depois de eventos como enchentes, incêndios, deslizamentos, apagões e falhas de comunicação.

Enquanto o Escopo do Produto apresenta o problema, a justificativa e a proposta da solução, este documento organiza a estrutura funcional do sistema. Para isso, são definidos os requisitos funcionais, requisitos não funcionais, regras de negócio, épicos, histórias de usuário, prioridades e organização em Kanban.

O objetivo do backlog é transformar a ideia do Orbital Memory em funcionalidades claras e organizadas, facilitando o planejamento, a modelagem e o desenvolvimento do protótipo. Dessa forma, o projeto mantém coerência entre a proposta conceitual e as entregas práticas das demais disciplinas, como interface web, interatividade com JavaScript, análise de risco em Python e simulação com Arduino.

4. Introdução

O desenvolvimento de um produto digital exige mais do que uma boa ideia. Para que uma solução seja viável, é necessário entender quais funcionalidades precisam ser construídas, quais regras devem ser respeitadas e quais requisitos garantem qualidade, segurança, usabilidade e coerência com o problema proposto.

No contexto do Orbital Memory, o backlog tem a função de organizar o desenvolvimento da plataforma a partir da visão dos usuários. O sistema foi pensado para apoiar órgãos como Defesa Civil, prefeituras, centros de monitoramento urbano e equipes de emergência em situações de desastre natural.

A proposta do produto envolve o monitoramento de dados urbanos, a classificação de níveis de risco, o registro de eventos críticos e a preservação de informações importantes em uma camada inspirada em infraestrutura orbital. Por isso, o backlog foi dividido em módulos, chamados de épicos, que representam as principais áreas do sistema.

Este documento apresenta a análise de requisitos e o planejamento inicial do produto, servindo como base para o desenvolvimento das demais entregas da Global Solution.

5. Visão geral do produto

O **Orbital Memory** é uma plataforma digital de monitoramento e preservação de dados críticos urbanos. Seu objetivo é apoiar cidades em situações de desastre natural, mantendo registros importantes mesmo quando parte da infraestrutura local está comprometida.

A solução funciona a partir da coleta de dados simulados de sensores urbanos, como nível de rio, temperatura, umidade, integridade da estação e status de comunicação. Esses dados são analisados pelo sistema, que classifica o nível de risco como baixo, médio, alto ou crítico.

Quando um evento relevante é identificado, ele é registrado no histórico da plataforma. O operador pode acompanhar os dados por meio de um dashboard, visualizar alertas, consultar ocorrências anteriores e simular a sincronização dos dados com uma camada de redundância inspirada em sistemas espaciais.

O produto será representado por meio de diferentes protótipos:

- dashboard web com HTML e CSS;
- interatividade com JavaScript;
- lógica de classificação de risco em Python;
- simulação física com Arduino e Tinkercad;
- apresentação em pitch.

A organização do backlog permite que todas essas partes sejam desenvolvidas de forma conectada, mantendo o projeto como uma solução única e integrada.

6. Requisitos funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer. Eles representam as funcionalidades principais esperadas para o Orbital Memory.

Código	Requisito funcional	Descrição
RF01	Visualizar sensores ativos	O sistema deve permitir que o operador visualize a quantidade de sensores ativos no monitoramento urbano.
RF02	Exibir nível de risco da cidade	O sistema deve mostrar o risco atual classificado como baixo, médio, alto ou crítico.
RF03	Registrar eventos críticos	O sistema deve registrar eventos como enchente, incêndio, falha de comunicação e instabilidade de sensores.
RF04	Exibir painel de alertas	O sistema deve apresentar alertas visuais para situações de atenção, risco alto ou risco crítico.
RF05	Consultar histórico de eventos	O sistema deve permitir a visualização de eventos registrados anteriormente.
RF06	Simular enchente	O sistema deve permitir a simulação de uma enchente, alterando os dados exibidos no dashboard.
RF07	Simular incêndio	O sistema deve permitir a simulação de um incêndio, atualizando temperatura, status e alertas.
RF08	Simular falha de comunicação	O sistema deve permitir a simulação de perda ou instabilidade de comunicação.
RF09	Simular sincronização orbital	O sistema deve exibir o status de preservação ou sincronização dos dados críticos.
RF10	Classificar risco com Python	O sistema deve utilizar uma lógica computacional em Python para classificar riscos com base em dados simulados.
RF11	Gerar relatório de ocorrência	O sistema deve gerar uma saída textual ou relatório simples com os eventos identificados.

RF12	Monitorar estação física simulada	O sistema deve ter uma simulação embarcada com Arduino, sensores, display e atuadores.
RF13	Acionar alerta automático	A estação simulada deve acionar LEDs ou buzzer automaticamente quando o risco for elevado.
RF14	Exibir telemetria local	A estação física simulada deve apresentar informações em display LCD.
RF15	Restaurar estado normal do sistema	A interface deve permitir retornar os dados simulados ao estado operacional normal.

7. Requisitos não funcionais

Os requisitos não funcionais indicam características de qualidade que o sistema deve apresentar, como desempenho, organização, usabilidade e confiabilidade.

Código	Requisito não funcional	Descrição
RNF01	Usabilidade	A interface deve ser clara, objetiva e fácil de interpretar por operadores em situações de risco.
RNF02	Acessibilidade visual	O sistema deve utilizar cores e contrastes que facilitem a leitura dos alertas e indicadores.
RNF03	Responsividade	O dashboard deve se adaptar a diferentes tamanhos de tela, sempre que possível.
RNF04	Organização do código	Os arquivos HTML, CSS, JavaScript, Python e Arduino devem estar organizados e comentados.
RNF05	Tempo de resposta	As interações do dashboard devem atualizar os dados de forma rápida durante as simulações.
RNF06	Confiabilidade	O sistema deve evitar informações ambíguas, principalmente nos alertas críticos.
RNF07	Clareza na documentação	Os repositórios devem conter README com explicações sobre uso, funcionalidades e execução.
RNF08	Coerência visual	A identidade visual deve transmitir tecnologia, segurança, missão crítica e conexão espacial.
RNF09	Segurança conceitual dos dados	O histórico de eventos deve ser tratado como informação crítica e preservada.
RNF10	Funcionamento autônomo na simulação embarcada	A estação Arduino deve responder aos sensores de forma automática, sem intervenção manual no código.

RNF11	Não bloqueio da simulação embarcada	O código Arduino deve evitar delay e utilizar lógica baseada em millis.
RNF12	Facilidade de demonstração	O protótipo deve ser simples de apresentar e entender durante a avaliação.

8. Regras de negócio

As regras de negócio definem condições que precisam ser respeitadas para que o sistema funcione de acordo com sua proposta.

Código	Regra de negócio	Descrição
RN01	Eventos críticos não devem ser apagados	Todo evento classificado como crítico deve permanecer registrado no histórico.
RN02	Risco crítico deve gerar alerta visual	Quando o risco for crítico, o sistema deve destacar o alerta de forma clara.
RN03	Sensores instáveis devem ser sinalizados	Quando um sensor apresentar comportamento anormal ou falha, ele deve ser marcado como instável.
RN04	A classificação de risco deve seguir os dados recebidos	O nível de risco deve ser calculado com base nos indicadores simulados.
RN05	Alertas devem indicar o tipo de evento	Todo alerta deve informar se está relacionado a enchente, incêndio, comunicação ou sensor.
RN06	O histórico deve conter horário, tipo, região e status	Cada evento registrado deve conter informações mínimas para consulta posterior.
RN07	O operador deve conseguir identificar rapidamente o estado da cidade	A interface deve priorizar indicadores principais no dashboard.
RN08	A sincronização orbital deve representar preservação dos dados	O status orbital simboliza a redundância e proteção das informações críticas.
RN09	A estação física deve atuar automaticamente	A simulação com Arduino deve acionar alertas sem depender de comando externo.
RN10	O sistema deve permitir retorno ao estado normal	Após uma simulação, deve ser possível restaurar os dados para uma situação estável.

9. Épicos do sistema

Os épicos representam grandes módulos ou grupos de funcionalidades do Orbital Memory.

Épico 1 — Monitoramento urbano

Este módulo concentra as funcionalidades relacionadas à visualização dos dados principais da cidade. Ele permite acompanhar sensores ativos, nível de risco, regiões monitoradas e status geral do sistema.

Funcionalidades relacionadas:

- visualizar sensores ativos;
- acompanhar risco atual;
- exibir status da cidade;
- identificar regiões em atenção.

Épico 2 — Alertas emergenciais

Este módulo reúne as funcionalidades ligadas à geração e visualização de alertas. Seu objetivo é informar rapidamente quando um evento de risco é identificado.

Funcionalidades relacionadas:

- exibir alertas críticos;
- simular enchente;
- simular incêndio;
- simular falha de comunicação;
- destacar alertas por nível de risco.

Épico 3 — Histórico de eventos

Este módulo representa a função de “memória urbana” do sistema. Ele registra eventos importantes e permite consulta posterior.

Funcionalidades relacionadas:

- registrar ocorrências;
- consultar eventos anteriores;
- armazenar horário, tipo, região e status;
- gerar base para relatórios.

Épico 4 — Redundância orbital

Este módulo representa a camada de preservação dos dados críticos inspirada em infraestrutura espacial.

Funcionalidades relacionadas:

- exibir status de sincronização orbital;
- indicar dados preservados;
- simular latência de comunicação;
- representar redundância de dados.

Épico 5 — Análise computacional de risco

Este módulo representa a lógica de classificação de risco desenvolvida em Python.

Funcionalidades relacionadas:

- receber dados simulados;
- calcular nível de risco;
- classificar eventos;
- gerar alertas e relatórios.

Épico 6 — Estação física simulada

Este módulo representa a simulação com Arduino no Tinkercad, funcionando como uma estação local do Orbital Memory.

Funcionalidades relacionadas:

- ler sensores simulados;
- exibir dados no LCD;
- acionar LEDs;
- ativar buzzer;
- atuar automaticamente em risco elevado.

10. Histórias de usuário

As histórias de usuário descrevem as funcionalidades do sistema a partir da necessidade de quem irá utilizá-lo.

Código	História de usuário	Prioridade
HU01	Como operador da Defesa Civil, quero visualizar o nível de risco da cidade para identificar rapidamente situações críticas.	Alta
HU02	Como operador da Defesa Civil, quero receber alertas visuais para agir com rapidez em caso de enchente, incêndio ou falha de comunicação.	Alta
HU03	Como gestor público, quero consultar o histórico de eventos para analisar ocorrências anteriores e melhorar o planejamento urbano.	Alta
HU04	Como operador do sistema, quero visualizar sensores ativos para saber se o monitoramento está funcionando corretamente.	Alta
HU05	Como operador do sistema, quero identificar sensores instáveis para evitar decisões baseadas em dados incorretos.	Média
HU06	Como agente de emergência, quero saber a região afetada pelo evento para priorizar o atendimento.	Alta
HU07	Como gestor público, quero que eventos críticos sejam preservados para apoiar investigações futuras.	Alta
HU08	Como operador, quero simular uma enchente para testar a resposta do sistema.	Média
HU09	Como operador, quero simular uma falha de comunicação para verificar como o sistema informa instabilidade.	Média
HU10	Como operador, quero acompanhar a sincronização orbital para saber se os dados críticos foram preservados.	Média
HU11	Como estudante desenvolvedora, quero organizar o código e a documentação para facilitar a avaliação e manutenção do projeto.	Alta

HU12	Como avaliador, quero acessar instruções claras no README para entender como executar e testar o protótipo.	Alta
HU13	Como operador, quero visualizar os dados em uma interface objetiva para tomar decisões mais rapidamente.	Alta
HU14	Como equipe de monitoramento, quero receber alertas automáticos da estação física para detectar riscos sem intervenção manual.	Alta
HU15	Como usuário do sistema, quero restaurar o dashboard ao estado normal após uma simulação para testar outros cenários.	Baixa

11. Priorização do backlog

A priorização foi definida considerando a importância de cada funcionalidade para demonstrar a proposta do Orbital Memory dentro do prazo da Global Solution.

Prioridade	Itens do backlog	Justificativa
Alta	Dashboard principal, alertas, histórico, classificação de risco, simulação Arduino, README	São funcionalidades centrais para explicar o produto e atender às exigências das disciplinas.
Média	Simulações específicas, sincronização orbital, relatórios simples, indicação de sensores instáveis	Complementam a experiência e fortalecem a demonstração do protótipo.
Baixa	Filtros avançados, login, exportação completa de relatórios, integração real com APIs externas	São úteis, mas não essenciais para o protótipo inicial.

Backlog priorizado

Ordem	Item	Prioridade	Status inicial
1	Criar dashboard principal	Alta	A fazer
2	Criar cards de sensores, risco e eventos	Alta	A fazer
3	Criar painel de alertas	Alta	A fazer
4	Criar tabela de histórico de eventos	Alta	A fazer
5	Implementar simulação de enchente	Alta	A fazer
6	Implementar simulação de incêndio	Média	A fazer
7	Implementar simulação de falha de comunicação	Média	A fazer
8	Criar lógica Python de classificação de risco	Alta	A fazer
9	Criar relatório de alertas em Python	Média	A fazer

10	Montar circuito Arduino no Tinkercad	Alta	A fazer
11	Implementar display LCD na estação	Alta	A fazer
12	Implementar LEDs e buzzer	Alta	A fazer
13	Criar README dos repositórios	Alta	A fazer
14	Criar documentação do produto	Alta	Em desenvolvimento
15	Criar pitch final	Média	A fazer

12. Jornada do usuário

A jornada do usuário representa como um operador da Defesa Civil ou de um centro de monitoramento utilizaria o Orbital Memory durante uma situação de risco.

Etapa	Ação do usuário	Resposta do sistema
1	O operador acessa o dashboard do Orbital Memory.	O sistema exibe sensores ativos, risco atual, eventos críticos e status orbital.
2	O operador verifica o painel principal.	O sistema apresenta indicadores visuais de operação normal, atenção ou risco crítico.
3	Um sensor indica aumento no nível de risco.	O sistema atualiza o status e registra o evento no histórico.
4	O operador consulta o painel de alertas.	O sistema mostra tipo de evento, região afetada e nível de gravidade.
5	O operador acompanha a evolução do evento.	O dashboard atualiza dados simulados e mantém a ocorrência registrada.
6	A camada orbital é acionada conceitualmente.	O sistema indica sincronização ou preservação dos dados críticos.
7	Após o evento, o operador consulta o histórico.	O sistema apresenta os registros para análise e planejamento futuro.
8	O operador gera ou consulta o relatório.	O sistema organiza as informações registradas para apoio à tomada de decisão.

Essa jornada foi pensada para priorizar agilidade, clareza visual e preservação de dados em situações nas quais a tomada de decisão precisa ser rápida e baseada em informações confiáveis.

13. Kanban do projeto

O Kanban organiza as tarefas do projeto em três colunas principais: **A fazer**, **Em desenvolvimento** e **Concluído**.

A fazer	Em desenvolvimento	Concluído
Criar dashboard em HTML e CSS	Finalizar Escopo do Produto	Definir nome do projeto
Implementar JavaScript no dashboard	Organizar Backlog do Produto	Definir problema central
Criar simulação de enchente		Definir solução proposta
Criar simulação de incêndio		Definir público-alvo
Criar simulação de falha de comunicação		Definir ODS relacionados
Criar lógica Python de risco		Criar arquivo-base do projeto
Criar README do Python		Criar wireframe inicial
Montar circuito no Tinkercad		Criar estrutura de pastas
Escrever README do Arduino		
Resolver atividade de matemática		
Criar roteiro do pitch		
Gravar vídeo final		

Organização das tarefas por disciplina

Disciplina	Principais tarefas
Software & Total Experience Design	Escopo do Produto, Backlog do Produto, requisitos, histórias de usuário e Kanban.
Front-End Design	Construção da interface estática com HTML e CSS.

Web Development	Interatividade com JavaScript, DOM, eventos e temporizadores.
Computational Thinking With Python	Programa para cálculo e classificação de risco.
Edge Computing & Computer Systems	Simulação Arduino com sensores, display, LEDs e buzzer.
Differentiated Problem Solving	Resolução matemática com derivadas e Newton-Raphson.
Storytelling e Inspiração Empreendedora	Pitch em vídeo apresentando a solução.

14. Critérios de aceite

Os critérios de aceite definem quando uma funcionalidade pode ser considerada pronta.

Item	Critério de aceite
Dashboard principal	Deve exibir nome do sistema, cards de status, painel de alertas e histórico de eventos.
Cards de status	Devem mostrar sensores ativos, risco atual, eventos críticos e status orbital.
Painel de alertas	Deve exibir alertas com diferença visual entre normal, atenção e crítico.
Histórico de eventos	Deve registrar pelo menos horário, tipo de evento, região e status.
Simulação de enchente	Deve alterar visualmente o risco, adicionar evento ao histórico e gerar alerta.
Simulação de incêndio	Deve alterar indicadores e registrar evento correspondente.
Simulação de falha de comunicação	Deve indicar instabilidade e simular recuperação ou alerta.
Sincronização orbital	Deve representar visualmente a preservação ou atualização dos dados.
Código Python	Deve receber dados simulados, calcular risco e exibir resultado.
Relatório Python	Deve gerar uma saída com classificação e descrição do alerta.
Simulação Arduino	Deve ler sensores simulados e acionar LEDs ou buzzer automaticamente.
Display LCD	Deve exibir dados de telemetria ou status da estação.
README	Deve explicar problema, solução, como executar e como testar o protótipo.
Pitch	Deve apresentar problema, solução, protótipos e impacto em até 3 minutos.

15. Conclusão

O backlog do Orbital Memory organiza as funcionalidades necessárias para transformar a proposta do produto em um protótipo viável e integrado. A partir da definição dos requisitos funcionais, requisitos não funcionais, regras de negócio, épicos, histórias de usuário e critérios de aceite, é possível visualizar com mais clareza como o sistema será desenvolvido.

O documento também demonstra que o projeto não será construído como partes isoladas, mas como uma solução única que conecta documentação, interface, interatividade, análise computacional, simulação embarcada, matemática aplicada e apresentação empreendedora.

Dessa forma, o backlog contribui para o planejamento do desenvolvimento e fortalece a coerência do Orbital Memory como uma plataforma de monitoramento e preservação de dados críticos urbanos inspirada em conceitos da Indústria Espacial.

16. Referências

FIAP. **Global Solution 2026 — 1º Semestre: Indústria Espacial: O Código que Move o Universo.** Engenharia de Software — 1º ano — Semipresencial, 2026.

Disponível no documento oficial da Global Solution enviado pela instituição.

Acesso em: jun. 2026.

ONU Brasil. **Objetivos de Desenvolvimento Sustentável.**

Disponível em: <https://brasil.un.org/pt-br/sdgs>

Acesso em: jun. 2026.

NASA. **Earth Science Data.**

Disponível em: <https://science.nasa.gov/earth/data/>

Acesso em: jun. 2026.

NASA Earthdata. **Your Gateway to NASA Earth Observation Data.**

Disponível em: <https://www.earthdata.nasa.gov/>

Acesso em: jun. 2026.

Copernicus Data Space Ecosystem. **Explore Data.**

Disponível em: <https://dataspace.copernicus.eu/explore-data>

Acesso em: jun. 2026.

Copernicus Programme. **Access to Data.**

Disponível em: <https://www.copernicus.eu/en/access-data>

Acesso em: jun. 2026.

INPE. **TerraBrasilis — Plataforma de dados geográficos.**

Disponível em: <https://terrabrasilis.dpi.inpe.br/>

Acesso em: jun. 2026.

INPE. **Instituto Nacional de Pesquisas Espaciais.**

Disponível em: <https://www.gov.br/inpe/pt-br>

Acesso em: jun. 2026.

INPE. **Programa Base de Informações Georreferenciadas.**

Disponível em: <https://data.inpe.br/>

Acesso em: jun. 2026.